

Vehicle Rental Management System

SB Car Rental



Item	Details
Course	Software Engineering
Instructor	Dr. Haya Samaaneh
Prepared by	Bana Sarhan & Shaimaa Ghanem
Team	SE-37

Introduction

This report presents the Software Quality Assurance activities performed on the Vehicle Rental Management System. The project focuses on improving software quality through static code analysis, refactoring, unit testing, code coverage measurement, and continuous integration.

SonarCloud was used to identify code smells, duplicated code, maintainability issues, and reliability problems. JaCoCo was used to measure code coverage, while JUnit 5 and Cucumber were used to implement automated testing. GitHub Actions was configured to automatically build, test, and analyze the project whenever new changes were pushed to the repository.

After applying the required improvements, the project achieved a significant increase in code coverage, a substantial reduction in duplicated code, and high ratings in maintainability, reliability, and security.

Project Overview

The Vehicle Rental Management System is a Java desktop application developed to simplify vehicle rental operations. The system allows users to manage vehicles, process rental and return operations, calculate rental costs, generate billing information, and send reminder notifications.

The project follows object-oriented programming principles and applies several software design patterns to improve flexibility and maintainability. Automated testing and static code analysis were performed to ensure software quality and compliance with the project requirements.

Abstract

This report presents the Software Quality Assurance activities carried out on the Vehicle Rental Management System. The project was analyzed using SonarCloud to identify code quality issues, while JUnit 5 and JaCoCo were used to improve test coverage and evaluate the quality of the code. Several code smells were fixed, duplicated code was reduced, and additional unit tests were implemented. As a result, the project achieved significant improvements in code coverage, maintainability, and overall software quality while maintaining high security and reliability ratings.

SonarCloud Analysis Before Refactoring

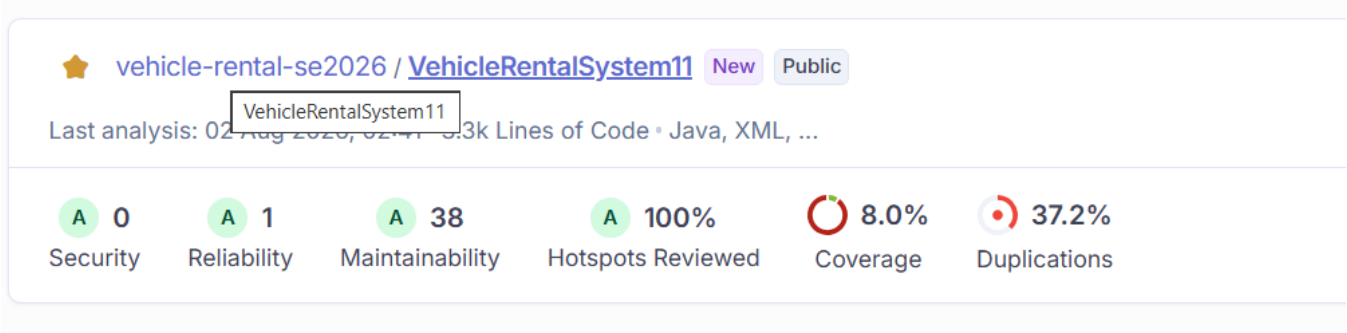


Figure 1. SonarCloud Analysis Before Refactoring

The initial SonarCloud analysis revealed several software quality issues. The project had a very low code coverage of 8.0%, indicating that only a small portion of the source code was tested. In addition, the duplication rate reached 37.2%, reflecting a high amount of repeated code, especially in the presentation layer. Although the project received good ratings for security and reliability, the overall quality needed improvement through additional testing and refactoring.

SonarCloud Analysis After Refactoring

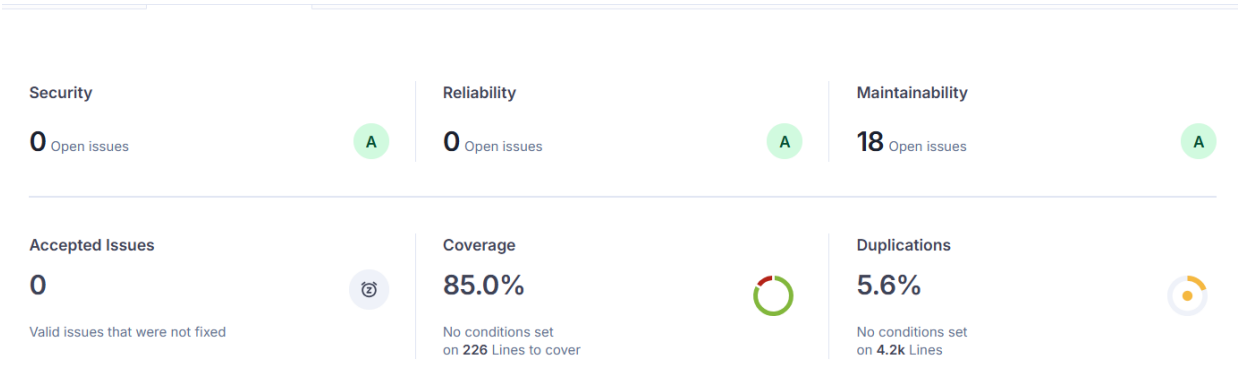


Figure 2. SonarCloud Analysis After Refactoring

After applying the required refactoring techniques and improving the unit tests, the overall software quality improved significantly. Additional JUnit test cases were implemented to increase the code coverage, while duplicated code was reduced by restructuring repeated code segments in the presentation layer. Several code smells reported by SonarCloud were also resolved, leading to better maintainability and overall project quality.

Metric	Before Refactoring	After Refactoring
Code Coverage	8.0%	85.0%
Code Duplications	37.2%	5.6%*
Security Rating	A	A
Reliability Rating	A	A
Maintainability Rating	A	A

- ✚ During the improvement process, we were able to increase the code coverage from 8.0% to 85.0% by adding more JUnit test cases. We also reduced code duplication by refactoring repeated code in the presentation layer. In addition, several code smells reported by SonarCloud were fixed, which improved the maintainability of the project. The project maintained an A rating in both Security and Reliability throughout the development process. Overall, these improvements made the system more reliable, easier to maintain, and significantly enhanced its overall software quality.
- ✚ The refactoring process and additional automated tests significantly enhanced the software quality. The project became easier to maintain, more reliable, and better tested, satisfying the main Software Quality Assurance objectives.

Code Smells Fixed

Code Smell 1: Replacing `System.out.println()` with `Logger`

Description

During the SonarCloud analysis, we noticed that `EmailNotificationService` was using `System.out.println()` to display reminder messages. SonarCloud reported this as a **Code Smell** because using `System.out.println()` is not considered a professional way to log application events. It makes log management difficult, especially in larger projects.

After reviewing the issue, we decided to replace `System.out.println()` with the Java **Logger** class. The Logger API provides a standard way to record application events and makes the code easier to maintain and debug in the future. Therefore, we created a `Logger` object and used `LOGGER.info()` instead of printing directly to the console.

Before

```
@Override
public void sendReminder(String message) {
    System.out.println("Reminder Email: " +
message);
}
```

After

```
private static final Logger LOGGER =

Logger.getLogger(EmailNotificationService.class.getN
ame());

@Override
public void sendReminder(String message) {
    LOGGER.info("Reminder Email: " + message);
}
```

Code Smell 2: Replacing System.out.println() in EmailObserver

Description

The same issue appeared in the EmailObserver class. The notification message was displayed using System.out.println(), which SonarCloud identified as a maintainability issue. Since this class is responsible for handling notification messages, using a proper logging mechanism is a better design choice.

To fix this issue, we replaced the console output with the Java Logger class. This change makes the implementation more consistent with Java best practices and improves the maintainability of the system.

Before

```
@Override
public void update(String message) {
    System.out.println("Email Notification: " +
message);
}
```

After

```
private static final Logger LOGGER =
Logger.getLogger(EmailObserver.class.getName());

@Override
public void update(String message) {
    LOGGER.info("Email Notification: " + message);
}
```

Code Smell 3: Replacing System.out.println() in InternalNotificationObserver

Description

We found the same Code Smell in the InternalNotificationObserver class. The class was printing notification messages directly to the console using System.out.println(). According to SonarCloud recommendations, logging should be handled using a dedicated logging framework rather than standard output.

To solve the problem, we replaced the console output with the Java Logger class. This modification keeps the logging implementation consistent across the project and follows Java coding standards.

Before

```
@Override
public void update(String message) {
    System.out.println("Internal Notification: " +
message);
}
```

After

```
private static final Logger LOGGER =

Logger.getLogger(InternalNotificationObserver.class.
getName());

@Override
public void update(String message) {
    LOGGER.info("Internal Notification: " +
message);
}
```

Test Coverage

8/2/26, 12:37 PM

VehicleRentalSystem11

[Sessions](#) VehicleRentalSystem11

VehicleRentalSystem11

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.vehiclerental.domain	65 81	55%	n/a	15	29 24	56 15	29	0 3
com.vehiclerental.service	36 163	81%	5 11	68%	10 24 14	63 6	16	1 4
com.vehiclerental.billing	10 30	75%	2	100%	2 7 5	14 2	6	0 1
com.vehiclerental.repository	7 190	96%	2	100%	2 11 3	33 2	10	0 3
com.vehiclerental.observer	6 46	88%	2	100%	1 9 2	16 1	8	0 3
com.vehiclerental.vehicletype	5 56	100%	4	100%	0 10 0	13 0	8	0 5
com.vehiclerental.enums	3 33	100%	n/a	0	1 0	6 0	1	0 1
com.vehiclerental.notification	8	100%	n/a	0	2 0	3 0	2	0 1
Total	124 of 731	83%	5 of 26	80%	30 93 48	204 26	80	1 21

Created with [JaCoCo](#) 0.8.12.202403310830

Figure 3. JaCoCo Test Coverage Report

To evaluate the quality of our test cases, we used JaCoCo to measure code coverage. We created unit tests for the main components of the system, including the service, repository, billing, observer, notification, vehicle type, and domain packages. After adding and improving the test cases, the overall instruction coverage reached 83%, while branch coverage reached 80%. These results indicate that most of the application's core functionality was successfully tested, increasing the reliability of the system.

UML Class Diagram

The UML class diagram illustrates the overall architecture of the Vehicle Rental Management System. It presents the main classes, interfaces, and relationships between different components of the system. The diagram also highlights the use of the Strategy and Observer design patterns, in addition to the layered architecture, which improves modularity, maintainability, and code organization.

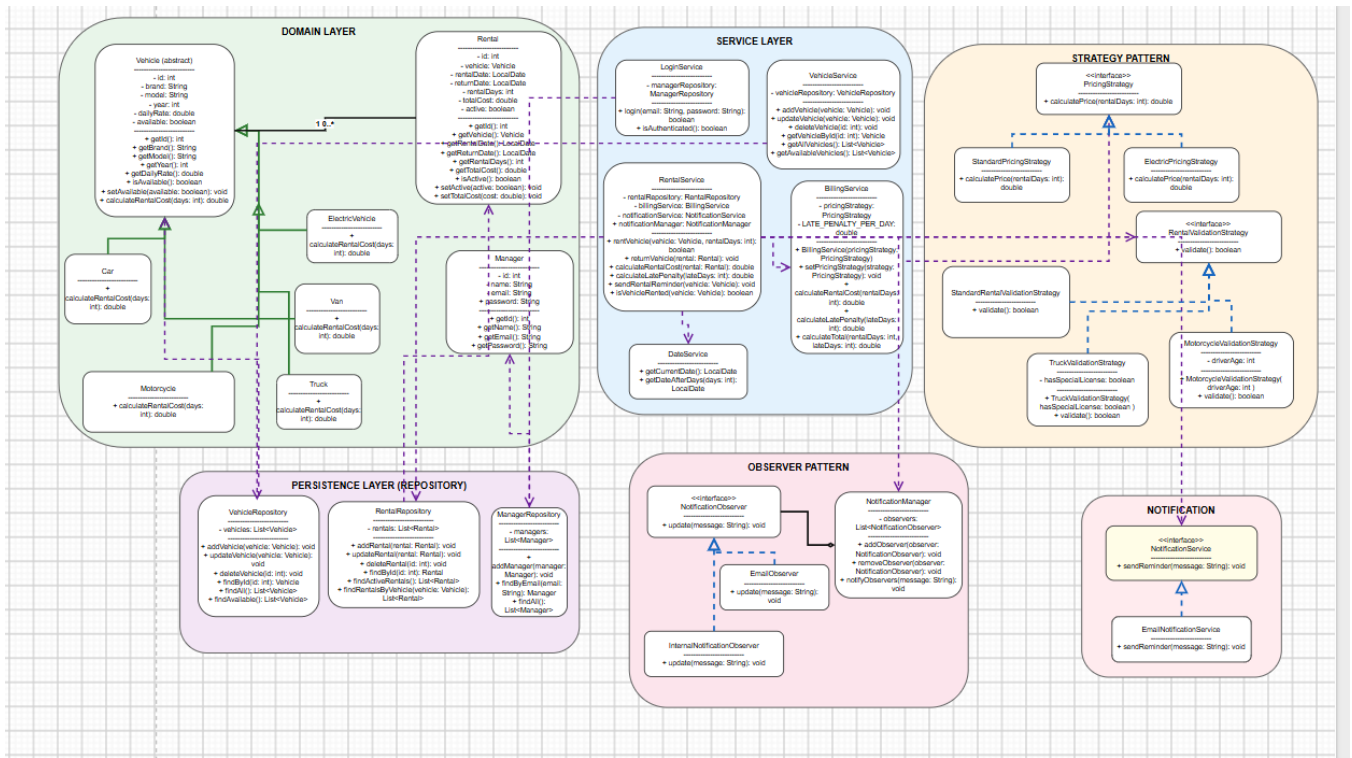


Figure 4. UML Class Diagram of the Vehicle Rental Management System

GitHub Repository

<https://github.com/vehicle-rental-se2026/Management-System.git>

The complete source code of the project, including unit tests, GitHub Actions workflow, and SonarCloud integration, is available in the GitHub repository.

Conclusion

This project demonstrated the importance of applying Software Quality Assurance techniques to improve the quality of a software system. Throughout the development process, we used SonarCloud to identify code smells and quality issues, JaCoCo to measure code coverage, and JUnit 5 to implement automated unit tests.

By refactoring the code, reducing duplication, and adding comprehensive test cases, we significantly improved the overall quality of the Vehicle Rental Management System. The code coverage increased from 8% to 85%, and the project achieved high ratings in Security, Reliability, and Maintainability.

Overall, this project helped us understand how software quality tools and testing practices contribute to building more reliable, maintainable, and well-structured applications. It also provided valuable hands-on experience in applying software engineering principles to a real-world project.